Introduction to Artificial Intelligence and Machine Learning

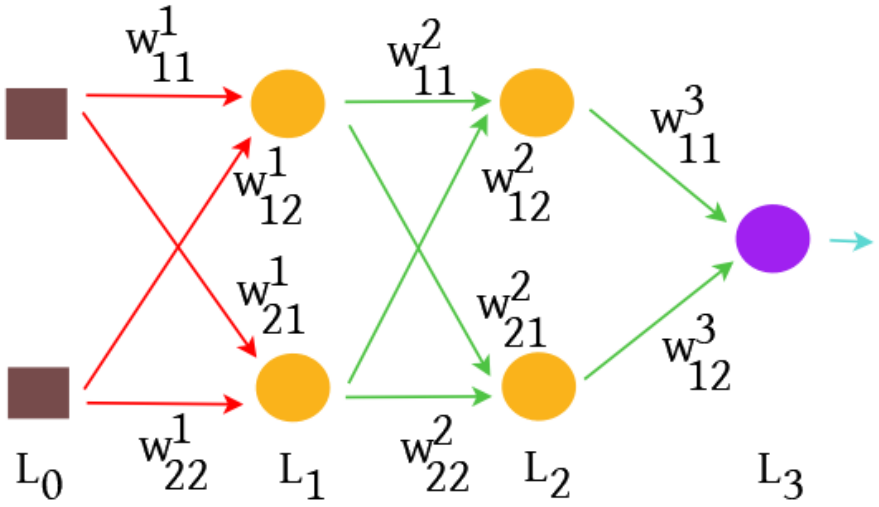
IE 206
Multi-layer Perceptrons

March 10, 2026.

1. Recap
   - MLP-Data Perspective

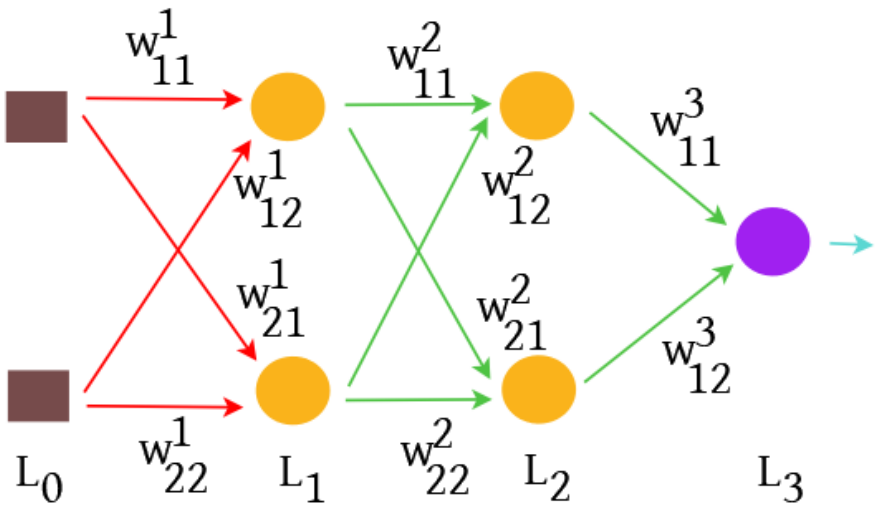
2. Optimization Concepts
   - Gradient Descent
   - Stochastic Gradient Descent
   - Mini-batch SGD

Multi Layer Perceptron - Data Perspective



- **Input:** Training Data $D = \{(x^s, y^s)\}_{s=1}^{S}$.
- For each sample x^s the prediction $\hat{y}^s = \text{MLP}(x^s)$.
- **Error:** $e^s = E(y^s, \hat{y}^s)$.
- **Aim:** To minimize $\sum_{s=1}^{S} e^s$.

Multi Layer Perceptron - Data Perspective

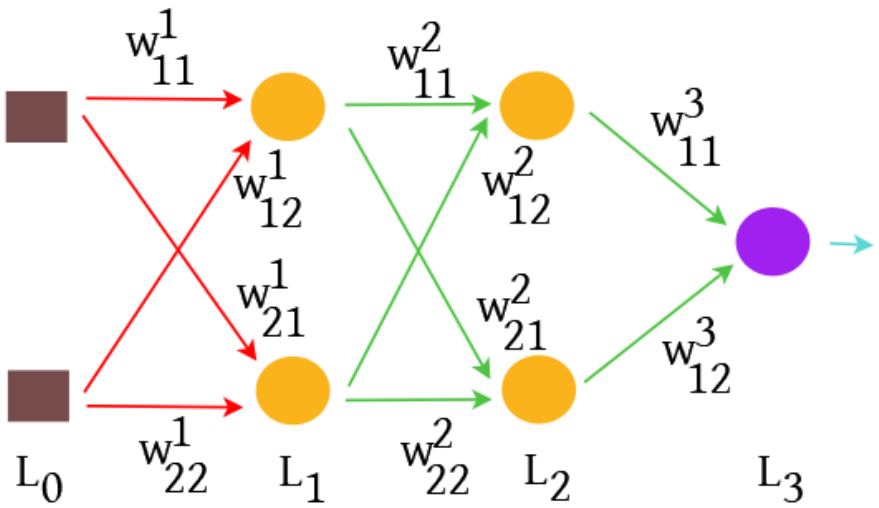


Optimization perspective

- Given training data $D = \{(x^s, y^s)\}_{s=1}^{S}$,

$$\min \sum_{s=1}^{S} e^s$$

Multi Layer Perceptron - Data Perspective

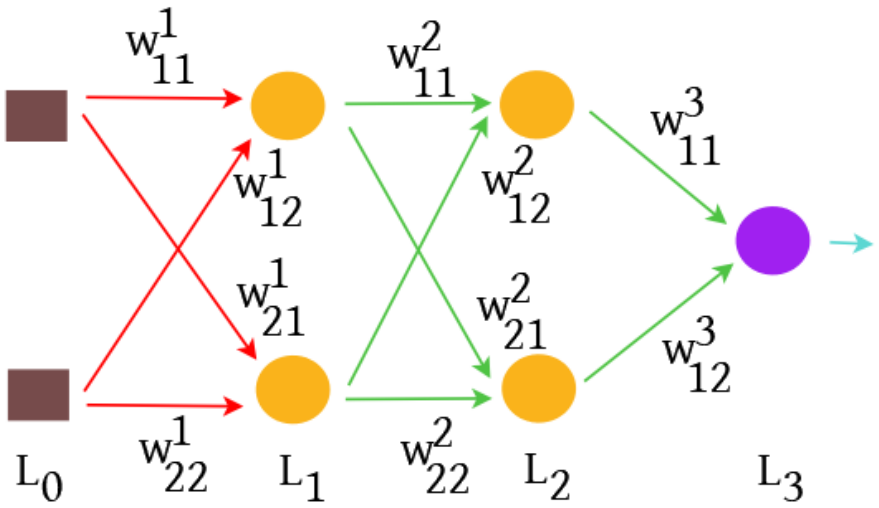


Optimization perspective

- Given training data $D = \{(x^s, y^s)\}_{s=1}^{S}$,

$$\min \sum_{s=1}^{S} e^s = \sum_{s=1}^{S} E(y^s, \hat{y}^s)$$

Multi Layer Perceptron - Data Perspective

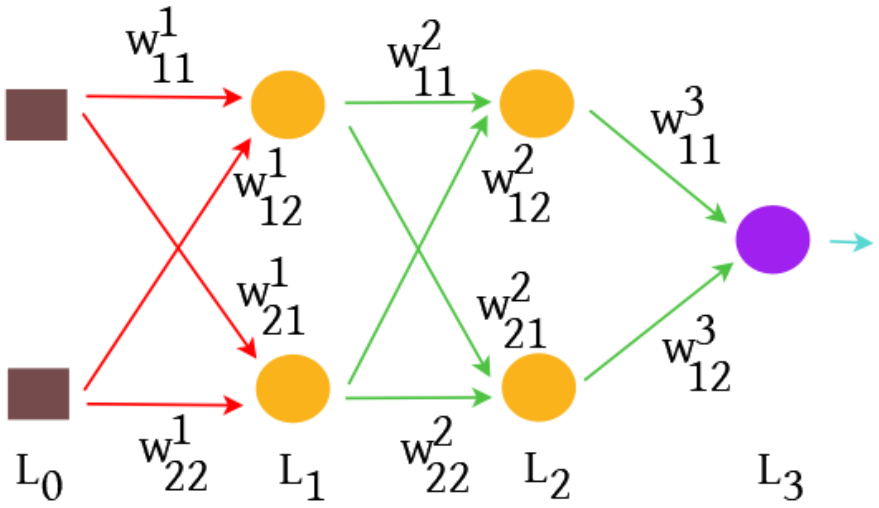


Optimization perspective

- Given training data $D = \{(x^s, y^s)\}_{s=1}^{S}$,

$$\min \sum_{s=1}^{S} e^s = \sum_{s=1}^{S} E(y^s, \hat{y}^s) = \sum_{s=1}^{S} E(y^s, \mathrm{MLP}(x^s))$$

Multi Layer Perceptron - Data Perspective



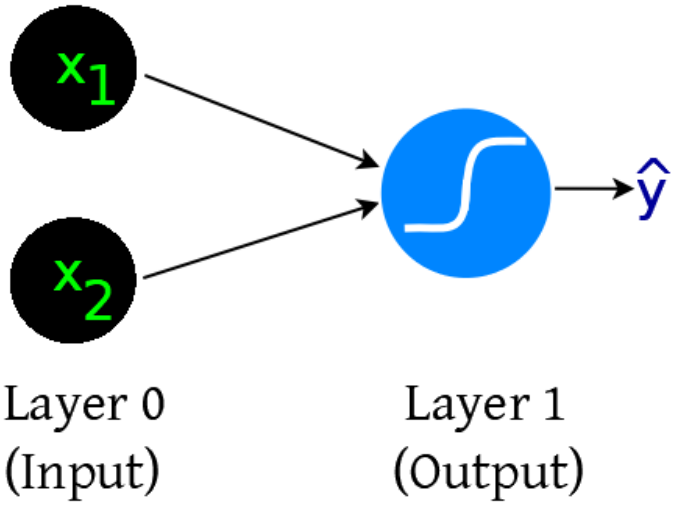
Optimization perspective

- Given training data $D = \{(x^s, y^s)\}_{s=1}^{S}$,

$$\min \sum_{s=1}^{S} e^s = \sum_{s=1}^{S} E(y^s, \hat{y}^s) = \sum_{s=1}^{S} E(y^s, \mathrm{MLP}(x^s))$$

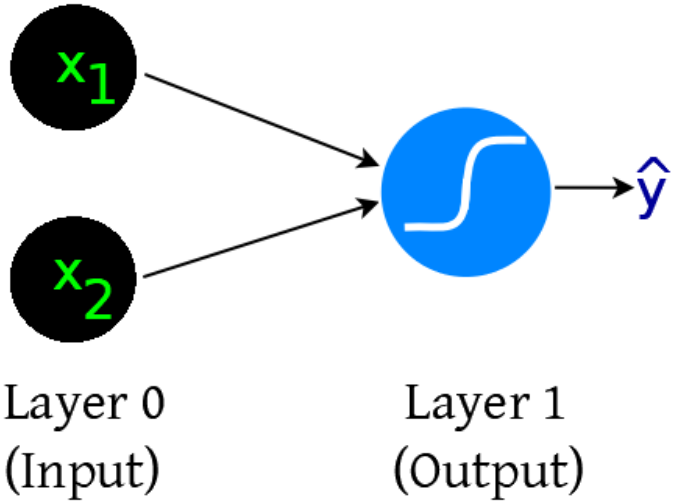
- Note: The minimization is over the weights of the MLP $W^1, \ldots, W^L$, where L denotes number of layers in MLP.

MLP - Data Perspective: A Simple Example



$$\hat{y} = \sigma(w_{11}^1 x_1 + w_{12}^1 x_2) = \frac{1}{1 + \exp\left(-[w_{11}^1 x_1 + w_{12}^1 x_2]\right)}$$

MLP - Data Perspective: A Simple Example

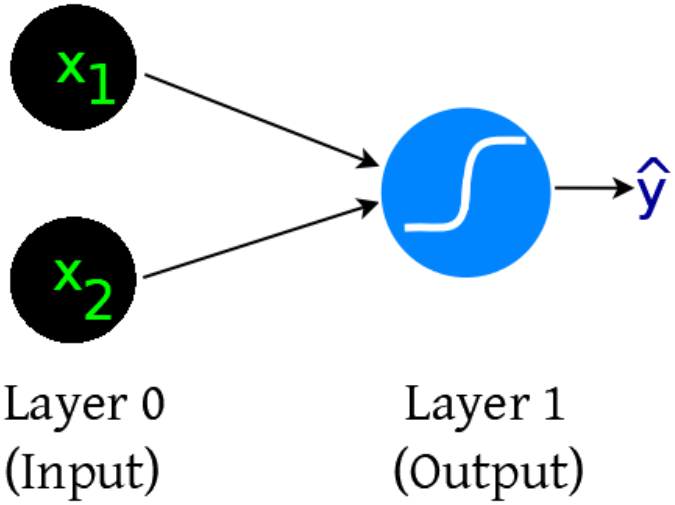


$$\hat{y} = \sigma(w_{11}^1 x_1 + w_{12}^1 x_2) = \frac{1}{1 + \exp\left(-[w_{11}^1 x_1 + w_{12}^1 x_2]\right)}$$

Property of 0-1 sigmoid $\sigma : \mathbb{R} \to [0, 1]$

- σ is continuous
- σ is monotonic
- $\sigma(z) \to \begin{cases} 0 & \text{if } z \to -\infty \\ 1 & \text{if } z \to +\infty \end{cases}$

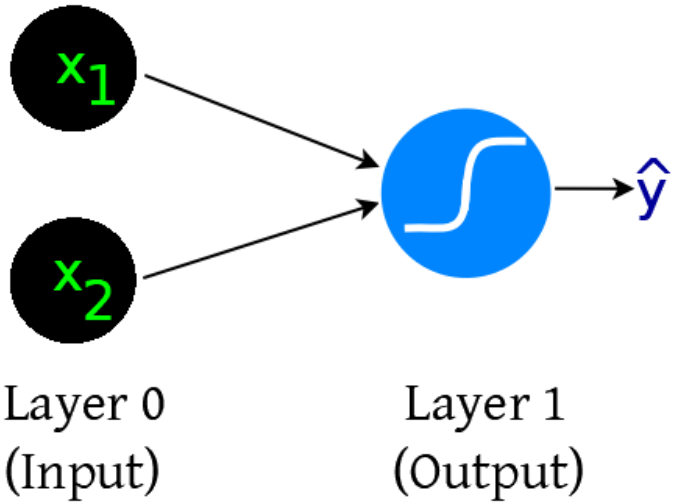
MLP - Data Perspective: A Simple Example



- Let

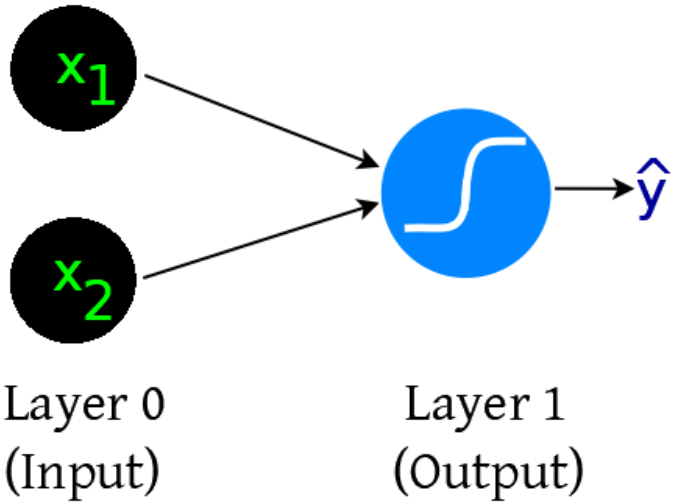
$$
\begin{aligned}
D = \{&(x^1 = (-3, -3), y^1 = 1),\\
&(x^2 = (-2, -2), y^2 = 1),\\
&(x^3 = (4, 4), y^3 = 0),\\
&(x^4 = (2, -5), y^4 = 0)\}.
\end{aligned}
$$

MLP - Data Perspective: A Simple Example



x_1	x_2	y	$\hat{y} = \sigma(w_{11}^1 x_1 + w_{12}^1 x_2)$
-3	-3	1	$\sigma(-3w_{11}^1 - 3w_{12}^1)$
-2	-2	1	$\sigma(-2w_{11}^1 - 2w_{12}^1)$
4	4	0	$\sigma(4w_{11}^1 + 4w_{12}^1)$
2	-5	0	$\sigma(2w_{11}^1 - 5w_{12}^1)$

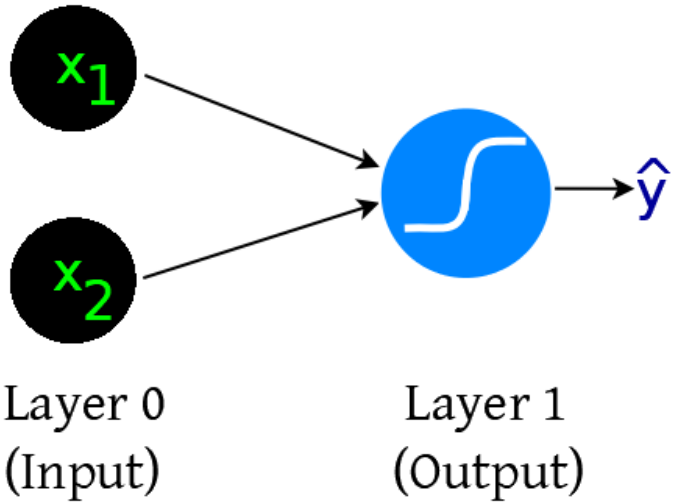
MLP - Data Perspective: A Simple Example



x_1	x_2	y	$\hat{y} = \sigma(w_{11}^1 x_1 + w_{12}^1 x_2)$
-3	-3	1	$\sigma(-3w_{11}^1 - 3w_{12}^1)$
-2	-2	1	$\sigma(-2w_{11}^1 - 2w_{12}^1)$
4	4	0	$\sigma(4w_{11}^1 + 4w_{12}^1)$
2	-5	0	$\sigma(2w_{11}^1 - 5w_{12}^1)$

- **Assume:** $\text{Err}(y, \hat{y}) = (y - \hat{y})^2$.

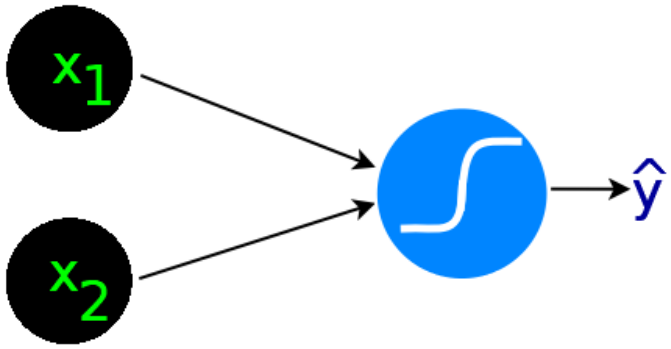
MLP - Data Perspective: A Simple Example



x_1	x_2	y	$\hat{y} = \sigma(w_{11}^1 x_1 + w_{12}^1 x_2)$
-3	-3	1	$\sigma(-3w_{11}^1 - 3w_{12}^1)$
-2	-2	1	$\sigma(-2w_{11}^1 - 2w_{12}^1)$
4	4	0	$\sigma(4w_{11}^1 + 4w_{12}^1)$
2	-5	0	$\sigma(2w_{11}^1 - 5w_{12}^1)$

- **Assume:** $\text{Err}(y, \hat{y}) = (y - \hat{y})^2$.
- Popularly called the squared error.

MLP - Data Perspective: A Simple Example



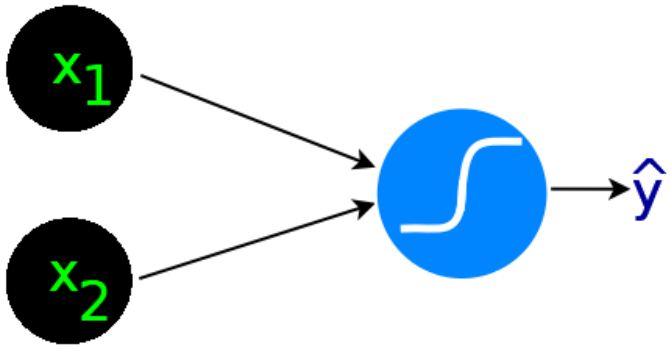
Layer 0 (Input) Layer 1 (Output)

x_1	x_2	y	$\hat{y} = \sigma(w_{11}^1 x_1 + w_{12}^1 x_2)$
-3	-3	1	$\sigma(-3w_{11}^1 - 3w_{12}^1)$
-2	-2	1	$\sigma(-2w_{11}^1 - 2w_{12}^1)$
4	4	0	$\sigma(4w_{11}^1 + 4w_{12}^1)$
2	-5	0	$\sigma(2w_{11}^1 - 5w_{12}^1)$

- Total error (or loss):

$$E = \sum_{i=1}^{4} e^i = \sum_{i=1}^{4} \mathrm{Err}(y^i, \hat{y}^i)$$

MLP - Data Perspective: A Simple Example



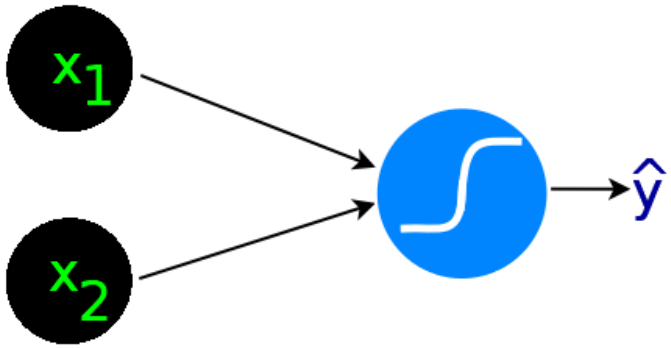
Layer 0 (Input) Layer 1 (Output)

x_1	x_2	y	$\hat{y} = \sigma(w_{11}^1 x_1 + w_{12}^1 x_2)$
-3	-3	1	$\sigma(-3w_{11}^1 - 3w_{12}^1)$
-2	-2	1	$\sigma(-2w_{11}^1 - 2w_{12}^1)$
4	4	0	$\sigma(4w_{11}^1 + 4w_{12}^1)$
2	-5	0	$\sigma(2w_{11}^1 - 5w_{12}^1)$

- Total error (or loss):

$$E = \sum_{i=1}^{4} \left(y^i - \frac{1}{1 + \exp\left(-\left[w_{11}^1 x_1^i + w_{12}^1 x_2^i\right]\right)} \right)^2$$

MLP - Data Perspective: A Simple Example



Layer 0 (Input) Layer 1 (Output)

x_1	x_2	y	$\hat{y} = \sigma(w_{11}^1 x_1 + w_{12}^1 x_2)$
-3	-3	1	$\sigma(-3w_{11}^1 - 3w_{12}^1)$
-2	-2	1	$\sigma(-2w_{11}^1 - 2w_{12}^1)$
4	4	0	$\sigma(4w_{11}^1 + 4w_{12}^1)$
2	-5	0	$\sigma(2w_{11}^1 - 5w_{12}^1)$

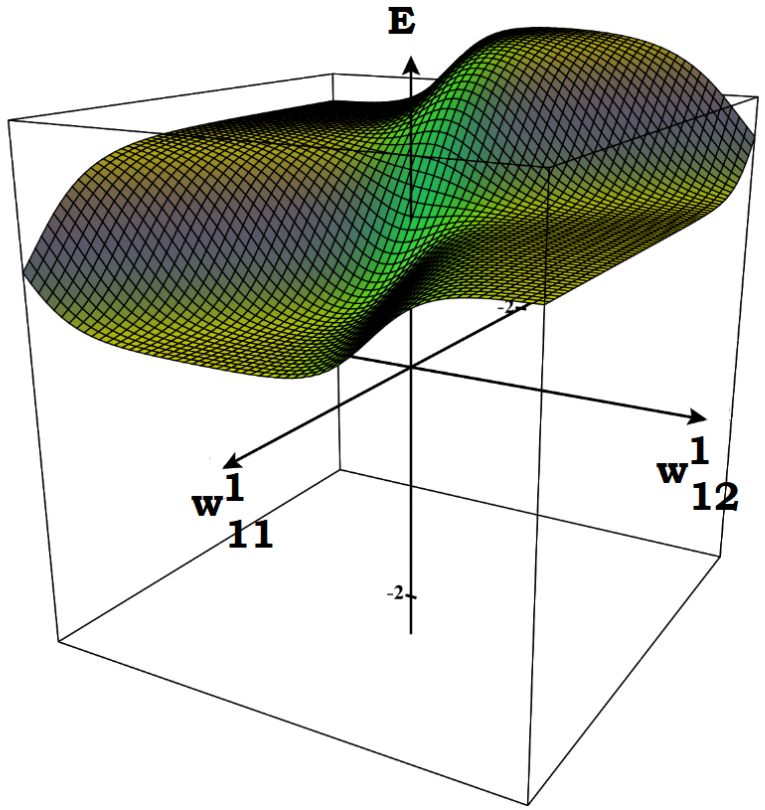
- Aim: To minimize the total error (or loss), which is

$$\min_{w_{11}^1, w_{12}^1} E = \sum_{i=1}^{4} \left(y^i - \frac{1}{1 + \exp\left(-\left[w_{11}^1 x_1^i + w_{12}^1 x_2^i\right]\right)} \right)^2$$

MLP - Data Perspective: A Simple Example

Visualizing the loss surface:

x_1	x_2	y	$\hat{y} = \sigma(w_{11}^1 x_1 + w_{12}^1 x_2)$
-3	-3	1	$\sigma(-3w_{11}^1 - 3w_{12}^1)$
-2	-2	1	$\sigma(-2w_{11}^1 - 2w_{12}^1)$
4	4	0	$\sigma(4w_{11}^1 + 4w_{12}^1)$
2	-5	0	$\sigma(2w_{11}^1 - 5w_{12}^1)$



$$E = \sum_{i=1}^{4} \left(y^i - \frac{1}{1 + \exp\left(-\left[w_{11}^1 x_1^i + w_{12}^1 x_2^i\right]\right)} \right)^2$$

Optimization Concepts

General Optimization Problem

$$\min_{x \in \mathcal{C}} f(x)$$

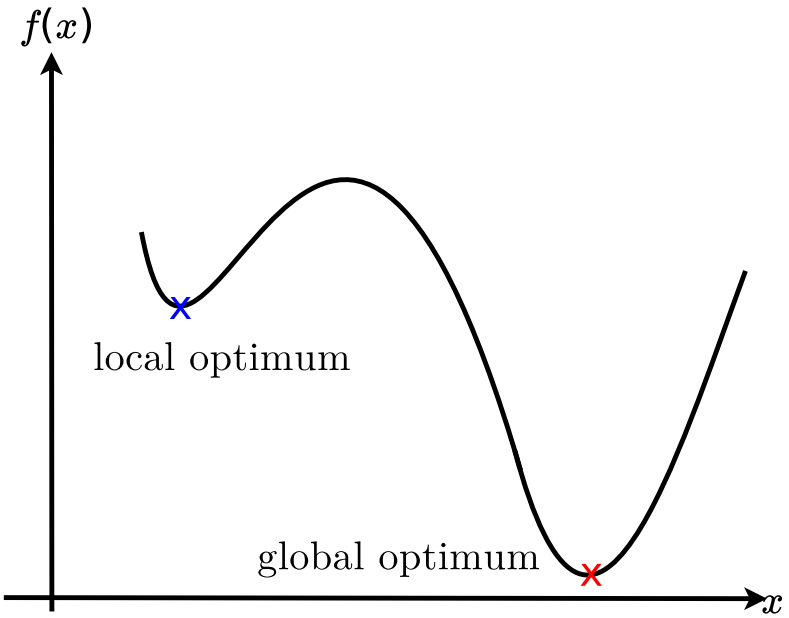
General Optimization Problem

$$\min_{x \in \mathcal{C}} f(x)$$

- f is called objective function and $\mathcal{C}$ is called feasible set.
- Let $f^* = \min_{x \in \mathcal{C}} f(x)$ denote the **optimal objective function value**.
- **Optimal Solution Set** $S^* = \{x \in \mathcal{C} : f(x) = f^*\}$.
- Let us denote by x^* an optimal solution in S^*.

General Optimization Problem

$$\min_{x \in \mathcal{C}} f(x) \tag{OP}$$



General Optimization Problem

$$\min_{x \in \mathcal{C}} f(x) \tag{OP}$$

Local Optimal Solution

A solution z to (OP) is called local optimal solution if $f(z) \leq f(\hat{z})$, $\forall \hat{z} \in \mathcal{N}(z, \epsilon)$ for some $\epsilon > 0$.

Note: $\mathcal{N}(z, \epsilon)$ denotes suitable $\epsilon-$neighborhood of z.

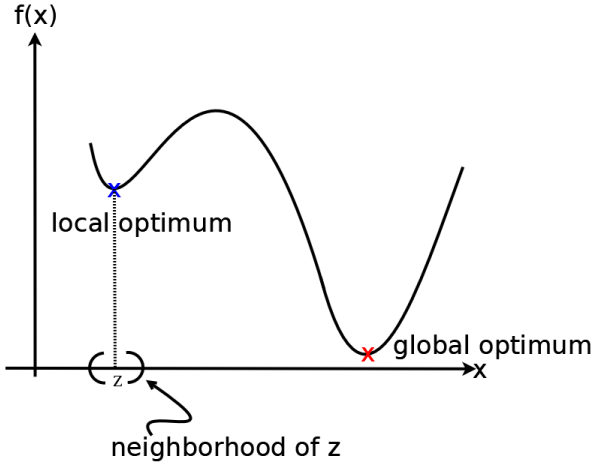
General Optimization Problem

$$\min_{x \in \mathcal{C}} f(x) \tag{OP}$$

Local Optimal Solution

A solution z to (OP) is called local optimal solution if $f(z) \leq f(\hat{z})$, $\forall \hat{z} \in \mathcal{N}(z, \epsilon)$ for some $\epsilon > 0$.

Note: $\mathcal{N}(z, \epsilon)$ denotes suitable $\epsilon-$neighborhood of z.



General Optimization Problem

$$\min_{x \in \mathcal{C}} f(x) \tag{OP}$$

Local Optimal Solution

A solution z to (OP) is called local optimal solution if $f(z) \leq f(\hat{z})$, $\forall \hat{z} \in \mathcal{N}(z, \epsilon)$ for some $\epsilon > 0$.

Note: $\mathcal{N}(z, \epsilon)$ denotes suitable $\epsilon-$neighborhood of z.

$\epsilon-$ Neighborhood of $z \in \mathcal{C}$

$$\mathcal{N}(z, \epsilon) = \{u \in \mathcal{C} : \text{dist}(z, u) \leq \epsilon\}.$$

General Optimization Problem

$$\min_{x \in \mathcal{C}} f(x) \tag{OP}$$

Local Optimal Solution

A solution z to (OP) is called local optimal solution if $f(z) \leq f(\hat{z})$, $\forall \hat{z} \in \mathcal{N}(z, \epsilon)$ for some $\epsilon > 0$.

Global Optimal Solution

A solution z to (OP) is called global optimal solution if $f(z) \leq f(\hat{z})$, $\forall \hat{z} \in \mathcal{C}$.

General Optimization Problem

$$\min_{x \in \mathcal{C}} f(x)$$

- **General Assumption:** $\mathcal{C} \subseteq \mathbb{R}^d$.

High Dimensional Representation - Notations

- Gradient of a function $f : \mathbb{R}^d \to \mathbb{R}$ at a point x

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \vdots \\ \vdots \\ \frac{\partial f(x)}{\partial x_d} \end{pmatrix}$$

General Optimization Problem

$$\min_{x \in \mathcal{C}} f(x)$$

- $\mathcal{C} \subseteq \mathbb{R}^d$.
- $f : \mathcal{C} \longrightarrow \mathbb{R}$.

Algorithm Development using Descent Direction

Consider the general optimization problem:

$$\min_{x \in \mathbb{R}^d} f(x) \qquad \text{(GEN-OPT)}$$

where $f : \mathbb{R}^d \longrightarrow \mathbb{R}$.

Gradient Descent Algorithm to solve (GEN-OPT)

- Start with $x^0 \in \mathbb{R}^d$.
- For $k = 0, 1, 2, \ldots$
  - If $\|\nabla f(x^{k+1})\|_2 = 0$, set $x^* = x^k$, break from loop.
  - $d^k = -\nabla f(x^k)$.
  - $\alpha^k = \text{argmin}_{\alpha > 0} f(x^k + \alpha d^k)$.
  - $x^{k+1} = x^k + \alpha^k d^k$.
- Output x^*.

Gradient Descent for our MLP Problem

Recall: For MLP, the loss minimization problem is:

$$\min_{w=(w_{11}^1, w_{12}^1)} E(w) = \sum_{i=1}^{4} e^i(w) = \sum_{i=1}^{4} \left(y^i - \frac{1}{1 + \exp\left(-\left[w_{11}^1 x_1^i + w_{12}^1 x_2^i\right]\right)} \right)^2$$

where $E : \mathbb{R}^2 \longrightarrow \mathbb{R}$.

Gradient Descent Algorithm to solve MLP Loss Minimization Problem

- Start with $w^0 \in \mathbb{R}^d$.
- For $k = 0, 1, 2, \ldots$
  - If $\|\nabla E(w^k)\|_2 = 0$, set $w^* = w^k$, break from loop.
  - $d^k = -\nabla E(w^k)$.
  - $\alpha^k = \text{argmin}_{\alpha > 0} E(w^k + \alpha d^k)$.
  - $w^{k+1} = w^k + \alpha^k d^k$.
- Output w^*.

Gradient Descent for our MLP Problem

Recall: For MLP, the loss minimization problem is:

$$\min_{w=(w_{11}^1, w_{12}^1)} E(w) = \sum_{i=1}^{4} e^i(w) = \sum_{i=1}^{4} \left(y^i - \frac{1}{1 + \exp\left(-\left[w_{11}^1 x_1^i + w_{12}^1 x_2^i\right]\right)} \right)^2$$

Gradient Descent Algorithm to solve MLP Loss Minimization Problem

- Start with $w^0 \in \mathbb{R}^d$.
- For $k = 0, 1, 2, \ldots$
  - If $\|\nabla E(w^k)\|_2 = 0$, set $w^* = w^k$, break from loop.
  - $d^k = -\nabla E(w^k)$.
  - $\alpha^k = \text{argmin}_{\alpha > 0} E(w^k + \alpha d^k)$.
  - $w^{k+1} = w^k + \alpha^k d^k$.
- Output w^*.

Gradient Descent for our MLP Problem

Recall: For MLP, the loss minimization problem is:

$$\min_{w=(w_{11}^1, w_{12}^1)} E(w) = \sum_{i=1}^{4} e^i(w) = \sum_{i=1}^{4} \left(y^i - \frac{1}{1 + \exp\left(-\left[w_{11}^1 x_1^i + w_{12}^1 x_2^i\right]\right)} \right)^2$$

Gradient Descent Algorithm to solve MLP Loss Minimization Problem

- Start with $w^0 \in \mathbb{R}^d$.
- For $k = 0, 1, 2, \ldots$
  - If $\|\nabla E(w^k)\|_2 = 0$, set $w^* = w^k$, break from loop.
  - $d^k = -\sum_{i=1}^{4} \nabla e^i(w^k)$.
  - $\alpha^k = \text{argmin}_{\alpha > 0} E(w^k + \alpha d^k)$.
  - $w^{k+1} = w^k + \alpha^k d^k$.
- Output w^*.

Stochastic Gradient Descent for our MLP Problem

Recall: For MLP, the loss minimization problem is:

$$\min_{w=(w_{11}^1, w_{12}^1)} E(w) = \sum_{i=1}^{4} e^i(w) = \sum_{i=1}^{4} \left(y^i - \frac{1}{1 + \exp\left(-\left[w_{11}^1 x_1^i + w_{12}^1 x_2^i\right]\right)} \right)^2$$

Stochastic Gradient Descent Algorithm to solve MLP Loss Minimization Problem

- Start with $w^0 \in \mathbb{R}^d$.
- For $k = 0, 1, 2, \ldots$
  - Choose a sample $j_k \in \{1, \ldots, 4\}$.
  - $w^{k+1} \leftarrow w^k - \gamma_k \nabla_w e^{j_k}(w^k)$.

Regularized Empirical Loss Minimization - Optimization Methods

Stochastic Gradient Descent Algorithm to solve MLP Loss Minimization Problem

- Start with $w^0 \in \mathbb{R}^d$.
- For $k = 0, 1, 2, \ldots$
  - Choose a sample $j_k \in \{1, \ldots, 4\}$.
  - $w^{k+1} \leftarrow w^k - \gamma_k \nabla_w e^{j_k}(w^k)$.

$\nabla_w e^{j_k}(w^k)$: Gradient at point w^k, of e^{i_k} with respect to w. Takes only $O(d)$ time.

Under suitable conditions on γ_k ($\sum_k \gamma_k^2 < \infty$, $\sum_k \gamma_k \to \infty$), this procedure converges **asymptotically**.
For smooth functions, $O(1/k)$ convergence possible (in theory!).
Typical choice: $\gamma_k = \frac{1}{k+1}$.

Mini-Batch Stochastic Gradient Descent for our MLP Problem

Mini-batch SGD Algorithm to solve MLP Loss Minimization Problem

- Start with $w^0 \in \mathbb{R}^d$.
- For $k = 0, 1, 2, \ldots$
  - Choose a block of samples $B_k \subseteq \{1, \ldots, 4\}$.
  - $w^{k+1} \leftarrow w^k - \gamma_k \sum_{j \in B_k} \nabla_w e^j(w^k)$.

Mini-batch Stochastic Gradient Descent for our MLP Problem

Mini-batch SGD Algorithm to solve MLP Loss Minimization Problem

- Start with $w^0 \in \mathbb{R}^d$.
- For $k = 0, 1, 2, \ldots$
  - Choose a block of samples $B_k \subseteq \{1, \ldots, 4\}$.
  - $w^{k+1} \leftarrow w^k - \gamma_k \sum_{j \in B_k} \nabla_w e^j(w^k)$.

- Restrictions on γ_k similar to that in SGD.
- **Asymptotic convergence** !

GD/SGD: Crucial Step

Recall: For MLP, the loss minimization problem is:

$$\min_{w=(w_{11}^1, w_{12}^1)} E(w) = \sum_{i=1}^{4} e^i(w) = \sum_{i=1}^{4} \left(y^i - \frac{1}{1 + \exp\left(-\left[w_{11}^1 x_1^i + w_{12}^1 x_2^i\right]\right)} \right)^2$$

Crucial step in Gradient Descent Algorithm

$w^{k+1} = w^k - \alpha^k \sum_{i=1}^{4} \nabla e^i(w^k)$

Crucial step in Stochastic Gradient Descent Algorithm

$w^{k+1} \leftarrow w^k - \gamma_k \nabla_w e^{j_k}(w^k)$.

Crucial step in Mini-batch SGD Algorithm

$w^{k+1} \leftarrow w^k - \gamma_k \sum_{j \in B_k} \nabla_w e^j(w^k)$.

GD/SGD for MLP: Crucial Step

Recall: For MLP, the loss minimization problem is:

$$\min_{w=(w_{11}^1, w_{12}^1)} E(w) = \sum_{i=1}^{4} e^i(w) = \sum_{i=1}^{4} \left(y^i - \frac{1}{1 + \exp\left(-\left[w_{11}^1 x_1^i + w_{12}^1 x_2^i\right]\right)} \right)^2$$

Crucial step in Gradient Descent Algorithm

$w^{k+1} = w^k - \alpha^k \sum_{i=1}^{4} \nabla e^i(w^k)$

Crucial step in Stochastic Gradient Descent Algorithm

$w^{k+1} \leftarrow w^k - \gamma_k \nabla_w e^{j_k}(w^k).$

Crucial step in Mini-batch SGD Algorithm

$w^{k+1} \leftarrow w^k - \gamma_k \sum_{j \in B_k} \nabla_w e^j(w^k).$

Note: $\nabla e^i(w^k)$, $\nabla_w e^{j_k}(w^k)$, $\nabla e^j(w^k)$ denote sample-wise gradient computation.